



UNIVERSIDADE FEDERAL DE SANTA CATARINA

Universidade Federal de Santa Catarina
INE5429 - Segurança da Informação

Implementação e Análise Crítica de um Pipeline DevSecOps

Sistema avaliado: TicketHub, Plataforma de Venda e Validação de Ingressos (.NET 8)

Leonardo Lima Appio - 21101963

Repositório: <https://github.com/leoappio/tickethub>

Sumário Executivo

Este trabalho projeta, implementa e analisa criticamente um pipeline completo de **DevSecOps** aplicado ao **TicketHub**, um sistema de autoria própria (.NET 8 / ASP.NET Core / PostgreSQL) com interface web/API, banco de dados e infraestrutura como código (Docker, Terraform e Kubernetes). O pipeline roda em **GitHub Actions** e contempla as cinco análises exigidas, Secret Detection, SCA, SAST, IaC Scanning e DAST, com ferramentas de mercado. O sistema foi instrumentado com fraquezas realistas; cada falha de risco médio ou superior foi corrigida, com a redução comprovada por nova execução das ferramentas:

Etapa	Ferramenta	Antes	Depois
Secret Detection	Gitleaks	10 segredos	0 no HEAD
SCA	Trivy / dotnet	3 CVEs HIGH	0
SAST	Semgrep (+ CodeQL)	9 (7 ERROR + 2 WARNING)	1 (falso positivo)
IaC Scanning	Trivy config / Checkov	25 misconfigs	4 (FP / risco aceito)
DAST	OWASP ZAP	1 High, 2 Medium, 7 Low	mitigados

1. Descrição do Sistema e Ferramental

1.1 Contexto de uso

O **TicketHub** é uma plataforma de bilheteria para eventos. Três perfis interagem com o sistema: **Organizer** (cria eventos e tipos de ingresso, publica eventos), **Customer** (pesquisa eventos, cria pedidos, paga e recebe ingressos com código único) e **Admin** (consulta relatórios e a base de usuários).

A lógica de negócio é não-trivial: controle de capacidade e estoque por tipo de ingresso (impede *overselling*), máquina de estados de pedido (Pending → Paid → Cancelled/Refunded), emissão de ingressos com código único após pagamento e validação no portão (ingresso de uso único). Há autenticação JWT, autorização por papéis/políticas e registro de tentativas de login.

1.2 Arquitetura e stack tecnológico

Arquitetura limpa (*Clean Architecture*) em quatro camadas, com o fluxo de dependências apontando sempre para o domínio:

```
TicketHub.Api           (apresentacao)  Controllers, JWT, Swagger, pagina publica HTML
-> TicketHub.Application (aplicacao)     Contratos de servico, DTOs, Token/PasswordService
    -> TicketHub.Infrastructure (infra)   EF Core + Npgsql, repositorios, implementacoes
        -> TicketHub.Domain             (dominio)  Entidades, enums, settings
```

Camada / aspecto	Tecnologia
Linguagem / runtime	C# / .NET 8
Framework web	ASP.NET Core 8 (Web API + página server-rendered)
ORM / banco	Entity Framework Core 8 / PostgreSQL 16
Autenticação	JWT Bearer (HMAC-SHA256) + autorização por papéis/políticas
Containerização	Docker (multi-stage) + docker-compose
IaC	Terraform (AWS) e Kubernetes
CI/CD	GitHub Actions

1.3 Plataforma de CI/CD e ferramental por análise

O pipeline está em `.github/workflows/devsecops.yml` e dispara em *push/pull_request*. Cada análise exigida foi mapeada a ferramentas consolidadas:

Análise	Ferramenta(s)	Por quê
Secret Detection	Gitleaks	Varre todo o histórico de commits (regex + entropia) buscando credenciais.
SCA	Trivy + dotnet list --vulnerable	Resolve o grafo de dependências NuGet (packages.lock.json) e cruza com bases de CVE.
SAST	Semgrep + CodeQL	Semgrep com packs p/csharp, p/security-audit, p/secrets e 5 regras customizadas; CodeQL para taint analysis profunda em C#.
IaC Scanning	Checkov + Trivy config	Cobrem Dockerfile, Terraform e Kubernetes com centenas de políticas de conformidade.
DAST	OWASP ZAP	Ataca a aplicação em execução (containerizada) buscando XSS, Injection, falhas de auth e cabeçalhos ausentes.

Como as regras *community* do Semgrep para C# não cobrem alguns *sinks* específicos (ex.: `FromSqlRaw`, `MD5.Create()`), foram escritas 5 regras customizadas em `.semgrep/tickethub-rules.yml`, estendendo o SAST ao contexto do projeto.

2. Evidências de Execução

Todas as ferramentas foram executadas sobre este repositório. Os artefatos brutos (JSON/SARIF/TXT) ficam em reports/01-before-fixes/ (estado vulnerável) e reports/02-after-fixes/ (pós-correção). Abaixo, os trechos mais relevantes.

2.1 Secret Detection, Gitleaks

```
$ gitleaks detect --source=. --report-format json --report-path reports/gitleaks.json
INF 6 commits scanned.
WRN leaks found: 10
```

10 segredos detectados (6 generic-api-key, 2 stripe-access-token, 2 aws-access-token) em appsettings.json, .env e docker-compose.yml. O arquivo .env foi adicionado e depois removido em commits distintos, o Gitleaks o encontra mesmo fora do HEAD, provando o valor da varredura sobre o histórico completo.

2.2 SCA, Trivy

```
$ trivy fs --scanners vuln --severity CRITICAL,HIGH,MEDIUM .
src/TicketHub.Infrastructure/packages.lock.json (nuget) Total: 3 (HIGH: 3)
Newtonsoft.Json 12.0.1 CVE-2024-21907 -> 13.0.1
System.Text.Json 8.0.4 CVE-2024-43485 -> 8.0.5
Microsoft.Extensions.Caching.Memory 8.0.0 CVE-2024-43483 -> 8.0.1
```

Três CVEs HIGH: um em dependência direta (Newtonsoft.Json) e dois transitivos trazidos pelo EF Core 8.0.8. O próprio NuGet corrobora com o aviso NU1903 em tempo de build.

2.3 SAST, Semgrep

```
$ semgrep scan --config=p/csharp --config=p/security-audit --config=p/secrets \
--config=.semgrep/tickethub-rules.yml --exclude=reports --exclude=bin .
Ran 78 rules on 53 files: 9 findings.
```

Sev.	Regra	Local
ERROR	efcore-fromsqlraw-injection	EventRepository.cs:30
ERROR	weak-password-hash-md5-sha1	PasswordService.cs:20
ERROR	hardcoded-secret-in-source	Program.cs:21, PasswordService.cs:16
ERROR	raw-html-response-xss	PublicController.cs:43
ERROR	detected-aws/stripe-key	appsettings.json:20,24
WARNING	permissive-cors-any-origin	Program.cs:56
WARNING	stacktrace-disclosure	Program.cs:88

2.4 IaC Scanning, Trivy config + Checkov

```
$ trivy config --severity CRITICAL,HIGH,MEDIUM .
Dockerfile Failures: 1 (HIGH 1) -> DS-0002 sem USER nao-root
k8s/deploy Failures: 8 (HIGH 3, MEDIUM 5) -> privileged, runAsRoot, sem limites
terraform Failures: 16 (CRIT 1, HIGH 11, MED 4) -> SG 0.0.0.0/0, RDS publico, S3 publico
```

Checkov corrobora: 32 falhas em Terraform, 21 em Kubernetes, 2 em Dockerfile e 3 de segredo em IaC. Exemplos de IDs: CKV_AWS_24 (SSH aberto), CKV_AWS_20 (S3 público), CKV_K8S_16 (container privilegiado).

2.5 DAST, OWASP ZAP

A etapa DAST sobe o stack completo (docker compose up) e executa o OWASP ZAP em varredura ativa alimentada pela especificação OpenAPI/Swagger da aplicação

(zap-api-scan.py), exercitando todos os endpoints, sem isso o spider só veria a raiz 404. Contra a versão vulnerável (tag v0-vulnerable-baseline) o ZAP reportou **1 alerta High, 2 Medium e 7 Low**:

```
$ zap-api-scan.py -t http://localhost:8080/swagger/v1/swagger.json -f openapi
[High]    SQL Injection /api/events/search?q=' (param q, 500 error-based)
[Medium]  Content Security Policy (CSP) Not Set /public/events?search=
[Medium]  Missing Anti-clickjacking Header /public/events?search=
[Low]     X-Content-Type-Options Header Missing /api/admin/users
[Low]     Cross-Origin-Resource-Policy Missing /api/admin/users
[Info]    User Controllable HTML (Potential XSS) /public/events?search=ZAP
WARN-NEW: 8      PASS: 113
```

Alerta ZAP	Endpoint	Fraqueza correspondente
SQL Injection (High)	/api/events/search?q=	FromSqlRaw com interpolação; payload q=' gera HTTP 500
User Controllable HTML / Potential XSS	/public/events?search=	eco da entrada refletido sem encoding
CSP / Anti-clickjacking / nosniff ausentes	global	sem CSP, X-Frame-Options, X-Content-Type-Options
X-Content-Type-Options Missing (em rota admin)	/api/admin/users	rota respondeu 200 sem autenticação (BAC)
Server Error response (info disclosure)	/api/events/search	erro 500 detalhado a partir da injeção

O **SQL Injection** foi confirmado dinamicamente: o payload q=' provoca HTTP 500 (injeção *error-based*) no buscador que usa FromSqlRaw. O endpoint /api/admin/users respondeu 200 sem autenticação (Broken Access Control). Após as correções (parametrização, encoding, [Authorize] e middleware de cabeçalhos) uma nova varredura não reproduz esses alertas. O relatório HTML completo está em reports/01-before-fixes/zap-before.html e é gerado automaticamente pelo job dast no GitHub Actions.

3. Análise de Falsos Positivos e Alertas Irrelevantes

Distinguir ruído de risco real é central em DevSecOps. Abaixo, alertas que **não** representam risco no contexto do TicketHub e a justificativa técnica para ignorá-los/aceitá-los.

3.1 Trivy AWS-0104 (CRITICAL), “egress irrestrito”

Após o hardening, o security group mantém uma regra de egresso: HTTPS (443) para 0.0.0.0/0. O Trivy classifica qualquer 0.0.0.0/0 de saída como crítico, mas tráfego HTTPS de saída para a internet é requisito legítimo (gateway de pagamento, APIs). **Risco aceito**, mitigado por limitar porta (443) e protocolo (TCP).

3.2 Trivy AWS-0132 (HIGH), “bucket sem chave KMS gerenciada pelo cliente”

O bucket já usa criptografia em repouso (`sse_algorithm = aws:kms`). O alerta exige uma CMK (customer-managed key); para os ativos do TicketHub a chave gerenciada pela AWS atende à política de dados. **Alerta irrelevante** ao contexto.

3.3 Trivy KSV-0125 (MEDIUM), “imagem de registry não confiável”

A regra marca `ghcr.io/...` como untrusted por não constar de uma allowlist padrão. O GitHub Container Registry é o registro oficial do projeto. **Falso positivo**.

3.4 Semgrep raw-html-response-xss após a correção (FP residual)

A regra customizada dispara sobre o padrão “resposta text/html montada manualmente”. Após a correção (codificação com `HtmlEncoder`), o XSS deixou de existir, mas o padrão sintático permanece, logo a regra continua acusando o arquivo. É um **falso positivo pós-remediação**: a heurística prioriza recall. Em produção, suprimir-se-ia com `// nosemgrep` justificado.

3.5 Segredos detectados dentro dos relatórios do próprio scanner

A primeira execução acusou segredos em `reports/gitleaks.json` e em `bin/Release/.../appsettings.json` (cópia de build). São artefatos de varredura e de compilação, não código-fonte. Corrigiu-se a configuração para excluir `reports/`, `bin/` e `obj/`, ruído clássico por escopo de varredura mal delimitado.

3.6 Checkov de hardening incremental (CKV_AWS_226, CKV_AWS_353, RDS IAM Auth)

Alertas remanescentes (auto-upgrade de minor, performance insights, autenticação IAM no RDS) são boas práticas operacionais, não vulnerabilidades exploráveis. Classificados como backlog de melhoria, ilustram que “falha de política” ≠ “vulnerabilidade”.

4. Identificação e Correção de Falhas Reais

As falhas abaixo são de risco médio ou superior, reais e exploráveis. Para cada uma: a fraqueza, o impacto e a correção (com diff). Os commits de correção estão entre as tags v0-vulnerable-baseline e v1-remediated.

4.1 SQL Injection no buscador público, CWE-89 (Crítico)

Deteção: Semgrep e ZAP. **Impacto:** o termo de busca era interpolado em SQL bruto via FromSqlRaw. No endpoint não autenticado /public/events?search=, um atacante exfiltra a base inteira, incluindo hashes de senha.

```
- var sql = $"... WHERE "Name" ILIKE '%{term}%' OR "Venue" ILIKE '%{term}%'";
- return await _db.Events.FromSqlRaw(sql).ToListAsync();
+ var pattern = $"%{term}%";
+ return await _db.Events.Where(e => e.IsPublished &&
+   (EF.Functions.ILike(e.Name, pattern) || EF.Functions.ILike(e.Venue, pattern)))
+   .AsNoTracking().ToListAsync();
```

A consulta passa a usar LINQ parametrizado: o termo viaja como valor de parâmetro, nunca como texto SQL.

4.2 Cross-Site Scripting na página pública, CWE-79 (Alto)

Deteção: Semgrep e ZAP. **Impacto:** PublicController concatenava a busca e os campos do evento em HTML (text/html) sem codificação, XSS refletido e armazenado. Permite roubo de sessão e ações em nome do usuário.

```
+ var enc = HtmlEncoder.Default;
- sb.Append("<p>Results for: ").Append(search);
+ sb.Append("<p>Results for: ").Append(enc.Encode(search));
- .Append(e.Name) ... .Append(e.Description)
+ .Append(enc.Encode(e.Name)) ... .Append(enc.Encode(e.Description))
```

4.3 Broken Access Control em /api/admin/users, CWE-862 + CWE-200 (Crítico)

Deteção: ZAP e revisão manual. **Impacto:** o endpoint não tinha [Authorize] e ainda retornava o PasswordHash de todos os usuários. Qualquer anônimo lia a base de credenciais.

```
+ [Authorize(Roles = "Admin")]
[HttpGet("users")] public async Task<IActionResult> Users() {
    var users = await _db.Users.AsNoTracking()
-   .Select(u => new { u.Id, u.Email, u.FullName, Role, u.PasswordHash, u.CreatedAt })
+   .Select(u => new { u.Id, u.Email, u.FullName, Role, u.CreatedAt })
```

4.4 Armazenamento de senha com MD5, CWE-916 (Alto)

Deteção: Semgrep. **Impacto:** senhas eram MD5(password + pepper), rápido e sem salt por usuário, vulnerável a rainbow tables e brute force em GPU.

```
- using var md5 = MD5.Create();
- return Convert.ToHexString(md5.ComputeHash(Encoding.UTF8.GetBytes(password + Pepper)));
+ var salt = RandomNumberGenerator.GetBytes(16);
+ var subkey = Rfc2898DeriveBytes.Pbkdf2(password, salt, 210_000, HashAlgorithmName.SHA256, 32);
+ return $"pbkdf2$210000${Convert.ToBase64String(salt)}${Convert.ToBase64String(subkey)}";
```

Migrou-se para PBKDF2-HMAC-SHA256, 210.000 iterações (diretriz OWASP 2023), salt aleatório de 128 bits por usuário e comparação em tempo constante.

4.5 Segredos hardcoded (código e configuração), CWE-798 (Crítico)

Deteção: Gitleaks, Semgrep, Checkov. **Impacto:** appsettings.json, .env e docker-compose.yml continham senha do Postgres, chave JWT e chaves Stripe/AWS/SendGrid; havia ainda uma chave JWT de fallback e um pepper embutidos no código.

```
- Key = "S3cr3t-JWT-Sign1ng-K3y-tickethub-2026-fallback" // fallback no Program.cs
+ if (string.IsNullOrEmpty(jwt.Key) || jwt.Key.Length < 32)
+     throw new InvalidOperationException("Jwt:Key deve vir de variável/secret store.");
```

Todos os segredos passam por variáveis de ambiente / secret store, com `.env.example` documentando o contrato. **Importante:** o Gitleaks confirma que os 10 segredos continuam no histórico git. A correção real exige **(1) rotacionar** imediatamente todas as chaves expostas (devem ser consideradas comprometidas) e **(2) reescrever o histórico** com `git filter-repo/BFG`. Apagar só do último commit dá falsa sensação de segurança.

4.6 Dependências vulneráveis, SCA (Alto)

Detecção: Trivy / dotnet. **Correção:** Newtonsoft.Json 12.0.1 → 13.0.3 e EF Core/Npgsql 8.0.8 → 8.0.11 (traz System.Text.Json 8.0.5 e Microsoft.Extensions.Caching.Memory 8.0.1 corrigidos). Nova varredura: **0 CVEs**.

4.7 Erros, CORS e cabeçalhos inseguros, CWE-16/942/209 (Médio)

```
- app.UseDeveloperExceptionPage(); // stack trace em producao
+ if (env.IsDevelopment()) app.UseDeveloperExceptionPage();
+ else { app.UseExceptionHandler("/error"); app.UseHsts(); }
- p.AllowAnyOrigin() // CORS para qualquer origem
+ p.WithOrigins(allowedOrigins)
+ // + middleware de headers: CSP, X-Frame-Options=DENY, nosniff, Referrer-Policy
```

4.8 Infraestrutura como Código, IaC (Crítico/Alto)

Recurso	Antes	Depois
Dockerfile	roda como root	USER 10001 não-privilegiado + HEALTHCHECK
Security Group	0.0.0.0/0 em 22/5432/8080; egress total	ingresso restrito a var.admin_cidr; egress só 443
RDS	público, sem criptografia, sem backup	privado, storage_encrypted, deletion_protection, backup 14d, senha no Secrets Manager
S3	ACL public-read, sem criptografia	block public access total, SSE-KMS, versionamento, logging
EC2	IMDSv1, IP público, disco sem cripto	IMDSv2 obrigatório, sem IP público, root volume criptografado
Kubernetes	privileged: true, runAsUser: 0, sem limites	runAsNonRoot, drop ALL caps, readOnlyRootFilesystem, limites, probes, Secret

Resultado pós-correção (Trivy config): **25** → **4** misconfigs, e as 4 restantes são os falsos positivos/riscos aceitos discutidos na Seção 3.

5. Conclusão

O pipeline cobre integralmente as cinco análises exigidas, integradas ao GitHub Actions e executadas sobre código de autoria própria com superfície de ataque real. Mais relevante que a contagem de alertas foi o processo de triagem: separar os falsos positivos/riscos aceitos das falhas reais, corrigi-las e comprovar a redução com nova execução das ferramentas (SAST 9→1, SCA 3→0, IaC 25→4). O caso dos segredos no histórico ilustra um princípio central de DevSecOps: a ferramenta aponta o sintoma, mas a correção segura exige entender o ciclo de vida completo do dado, aqui, rotacionar e expurgar, não apenas apagar.

Apêndice, Reprodução


```
# Análises estáticas (Secret, SCA, SAST, IaC):
bash scripts/run-all-scans.sh          # gera reports/
# DAST (requer Docker):
bash scripts/run-dast.sh                # sobe o stack + OWASP ZAP -> reports/zap-report.html
# Build e execução local:
cp .env.example .env && docker compose up --build  # http://localhost:8080/swagger

# Tags:  v0-vulnerable-baseline (antes)   |   v1-remediated (depois)
```